Database Management Systems
CPS 3740

Lecture 18

Computer Science, Kean University
Dr. Ching-yu (Austin) Huang
Instructor Paolien Wang

Outline

- Variables
- Temporary table
- Homework due soon
  - Seek help from Samurai ASAP if necessary

Status of homework

- Status
- Homework due soon: implement each requirement on our database server, then submit on class website.
  - Double-check the confirmation email
- Seek help ASAP - Code Samurai
- No late submission
  - No exception!!!

Variables in MySQL

- MySQL has three different kinds of variables – local, session, global.
- Local variables are set in the scope of a statement or block of statements.
  - Once that statement or block of statements has completed, the variable goes out of scope.
  - Using the DECLARE statement with a DEFAULT will set the value of a local variable.

    ex: DECLARE total INT DEFAULT 0;

MySQL Session variables

- Session variables are set in the scope of a session.
- A session starts with a connection to the server and ends when the connection is closed.
- Session variables go out of scope once the connection is terminated.
- Session variables created during a connection cannot be referenced from other sessions.
- To declare or reference a session variable, prefix the variable name with an **@** symbol:

```
SET @count = 100;
```

Session Variables Example

- @cost will go out of scope when the mysql connection is terminated.
- Values can be assigned to variables using the SET statement:

  SET @cost = @cost + 5.00;

```
> Set @x=10;
> Set @y=5;
> Set @z=@x + @y;
> Select @z;
> Select @x, @y;
Logout and login back
> Select @z;
```

Variables in MySQL – global variables

- Global variables exist across connections.
- They are set using the GLOBAL keyword: SET GLOBAL max_connections = 300;.
- Global variables are not self-defined, but are tied to the configuration of the running server.

Variables and values

- Values can be assigned to local, session, and global variables using the SET statement:

```
SET @cost = @cost + 5.00;
```

- MySQL's SET statement includes an extension that permits setting multiple variables in one statement:

```
SET @cost = 5, @cost1 = 8.00;
select @cost;
```

Declarations, User-Defined Variables

- Variables and constant variables must be declared before they can be referenced
- Possible to declare a variable as NOT NULL
- DECLARE total INT DEFAULT 0;
- User variables are written as @var_name
- One way to set a user-defined variable is by issuing a SET statement:
  - SET @var_name = expr;

Assignments

- For SET, either = or := can be used as the assignment operator.
- Variables can be assigned in two ways:
  - Using the normal assignment statement (:=):

    vStaffNo := ‘SG14’;

  - Using an SQL SELECT or FETCH statement:

```
SELECT COUNT(*) INTO x
FROM PropertyForRent
WHERE staffNo = vStaffNo;
```

Assignments

- Variables can be assigned in 3 ways:
  1. Using the normal assignment statement (:=):

     > SET @bno := 'B003';

  2. Using an SQL SELECT or FETCH statement:

  > SELECT COUNT(*) INTO @total FROM Staff WHERE branchno= @bno;

  3. Using SET

     > SET @total = (SELECT COUNT(*) FROM Staff);

- Using For SET, either = or := can be used as the assignment operator.

  SET @bno:= 'B003';

  SET @bno = 'B003';

Class exercise - Assignment (SELECT vs SET)

- You can also assign a value to a user variable in statements other than SET.
- In this case, the assignment operator must be := and not = because the latter is treated as the comparison operator = in non-SET statements:

mysql> SET @t1=1, @t2=2, @t3:=4;

mysql> SELECT @t1, @t2, @t3, @t3=4, @t3=8, @t4 := @t1+@t2+@t3;

@t1	@t2	@t3	@t3=4	@t3=8	@t4 := @t1+@t2+@t3
1	2	4	1	0	7

Class exercises - SELECT vs. SET

Example1:

```
mysql> SET @countTotal = (SELECT COUNT(*) FROM Staff);
```

OR

```
mysql> SELECT COUNT(*) INTO @countTotal FROM Staff;

mysql> select @countTotal;
```

Example2:

```
mysql> SET (@countTotal, @sum) = (SELECT COUNT(*) , sum(salary)FROM Staff);  => syntax error
```

OR

```
mysql> SELECT COUNT(*) INTO @countTotal, sum(salary) INTO @sum FROM Staff; => syntax error
```

OR

```
mysql> SELECT COUNT(*) , sum(salary) INTO @countTotal, @sum FROM Staff;
mysql> select @countTotal, @sum;
```

Exercises

- What is the results of the following SQL statements

mysql> SET @t1=1, @t3:=4;

mysql> SELECT @t1=2, @t4 := 2+@t3;

Logout and login back to database

mysql > SELECT @t1, @t3;

TEMPORARY TABLE

- In a database, a temporary table is a special type of table that allows us to store a temporary result set, which we can reuse several times in a single session.
- Temporary tables will be automatically removed when the session is closed or logout.
- A temporary table is very handy when it is impossible or expensive to query data that requires a single SELECT statement with JOIN clauses.
- Temporary tables can be used in stored procedures to store immediate result sets for the subsequent uses.
- Temporary table can be created by command: CREATE TEMPORARY TABLE and removed using the command: DROP TEMPORARY TABLE.

Create and Drop Temporary Table

- Create a temporary table

```
CREATE TEMPORARY TABLE tTest_demo
(id int, name varchar(10));
```

- Insert one record

```
INSERT INTO tTest_demo VALUES (1,'Test');
```

- SELECT the record
- Logout and login back
- SELECT the record again
- DROP the temporary table

```
DROP TEMPORARY TABLE tTest_demo;
```

Temporary Table Exercises

- Do these SQL statements in order

```
Drop temporary table tMax_table;

Create temporary table if not exists tMax_table as select fname,lname,salary from dreamhome.Staff where branchno='B003';

Select max(salary) into @mymax from tMax_table;

Drop temporary table tMax_table;

Select @mymax;   -- value is still in the system
```

Chapter 8

Advanced SQL
Stored Routines

Stored Routines

- PL/SQL includes procedural language elements such as conditions and loops.
- It allows declaration of constants and variables, procedures and functions, types and variables of those types, and triggers.
- It can handle exceptions (runtime errors).

Stored Routines

- PL/SQL (Procedural Language/Structured Query Language) is a block-structured language. It is not an object-oriented language.
- An SR is saved in the SQL server and perhaps compiled when the source code is saved.
- An SR runs, when called, in the context of the server, in one of the MySQL server process threads.

Stored Routines

- An SR could be simply seen as a set of SQL statements. Its body is a list of SQL statements.
- An SR has a name, may have a parameter list, and may contain a set of SQL statements.
- An SR is created using the CREATE command.

Stored Routines Types

```
CREATE PROCEDURE `DBName.mysp` (err VARCHAR(25))
.....
CALL mysp('...');
----------------------------------------

CREATE FUNCTION AddData()
RETURNS ........
SELECT AddData();
```

Stored Routines

- When an SR is invoked, an implicit USE DBName is performed (and undone when the SR terminates).
- USE statements within SRs are not allowed.
- When a DB is dropped, all the SRs associated with it are dropped as well.
- MySQL supports three types of routines:
  1. Stored Procedures (SP)
  2. Stored Functions (SF)
  3. Triggers (Graduate Database Course)

Stored Routine Types

- An Stored Procedure (SP) is a stored program that is called by using an explicit command (CALL). It does not return an explicit value.
- An Stored Function (SF) is a stored program that could be used as a user defined function and it is called by using just its name (without an explicit command). It can return a value that could be used in other SQL statements the same way it is possible to use pre-installed MySQL functions like pi(), replace(), etc.

Delimiter command

- delimiter //
- delimiter command to change the statement delimiter from ; to // while the procedure is being defined.
- This enables the ; delimiter used in the procedure body to be passed through to the server rather than being interpreted by mysql itself.
- Distinguish between the semicolon of the SQL statements in the procedure and the terminating character of the procedure definition
- Must have space between delimiter and //

Stored Routines

```
DELIMITER //
CREATE PROCEDURE `DB.proc_name`(name VARCHAR(25))
BEGIN
……
END;
//
DELIMITER ;
```

Stored Routine Parameter Types

- IN. This is the default (if not specified). It is passed to the routine and can be changed inside the routine, but remains unchanged outside.
- OUT. No value is supplied to the routine (it is assumed to be NULL), but it can be modified inside the routine, and it is available outside the routine.
- INOUT. The characteristics of both IN and OUT parameters. A value can be passed to the routine, modified there as well as passed back again.
- These parameter types are only for stored procedures, NOT for stored function. Why?

Stored Procedure Environment

- The information associated to an SP can be seen using the SHOW CREATE PROCEDURE command.
  - **show create procedure pTest_xxxx;**
- The information associated to an SP status can be seen using the SHOW PROCEDURE STATUS command.
  - **mysql> show procedure status where Name='pTest_demo' and Db like '%3740%';**
- The result of the last two statements can also be obtained using the information schema database and the routines table.

  SELECT * FROM INFORMATION_SCHEMA.ROUTINES WHERE ROUTINE_NAME='pTest_demo' and ROUTINE_SCHEMA like ‘%3740%';

Stored Functions

- An SF is a routine that could be used in other SQL statements the same way it is possible to use defined MySQL functions like length(), etc.
- An SF cannot display results.
- An SF is created using the CREATE command.
- The input parameters (IN) are not required in an SF.
- An SF must have a RETURN statement and can only return one value.

Stored Functions

- An SF it is called just by using its name.
- An SF name should be different than existing SQL functions, otherwise you must insert a space between the name and the parenthesis, when defining and calling it.

Stored Function Environment

- The environment where an SF is stored and could be examined is the server data dictionary.
- An SF is stored in a special table called mysql.proc within a database's namespace.
- The information associated to a SF could be seen using the SHOW CREATE FUNCTION command.
- The information associated to an SF status could be seen using the SHOW FUNCTION STATUS command.

Stored Function Environment

- The result of the last two statements can also be achieved using the information schema database and the ROUTINES table.

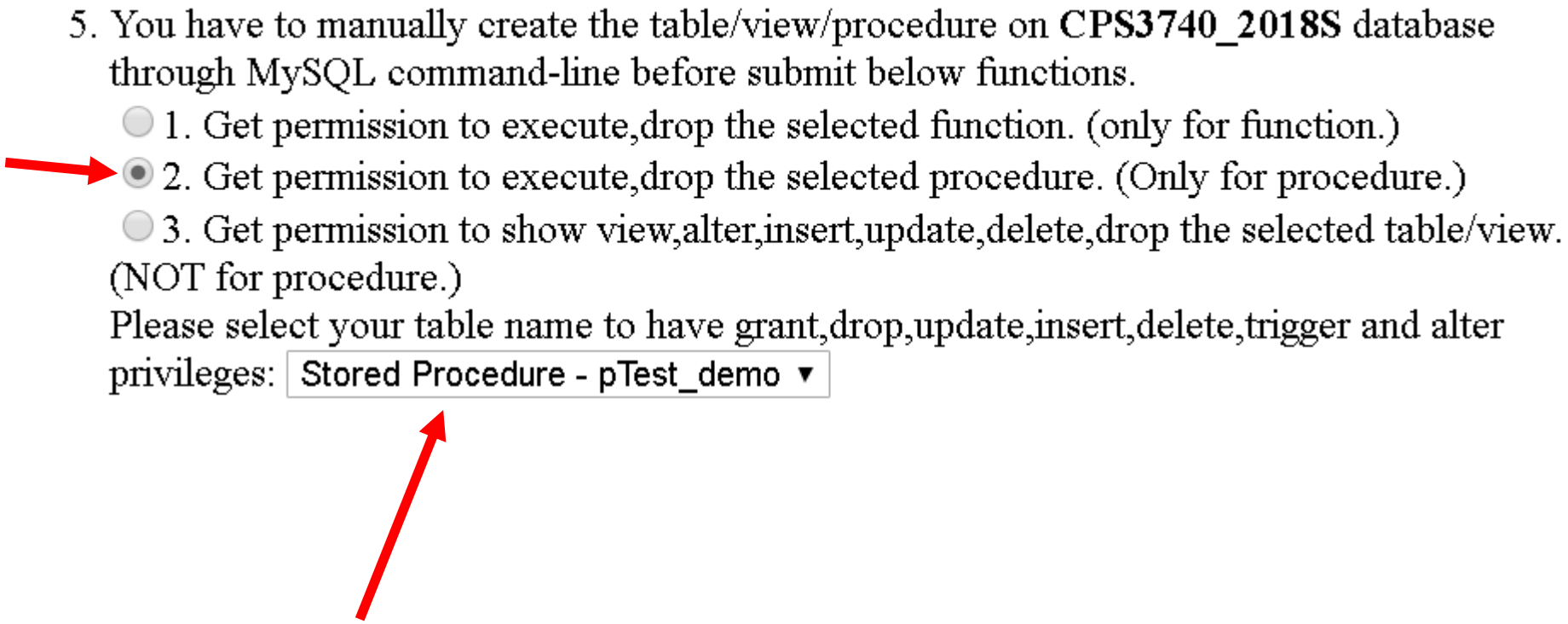
```
SELECT * FROM
INFORMATION_SCHEMA.ROUTINES WHERE
ROUTINE_NAME='SFName' ;
```

Functions

```
CREATE  FUNCTION  fTest_xxxxx (s CHAR(20))
   RETURNS  CHAR(50)
RETURN CONCAT('Hello, ',s,'!');
```

- The **RETURNS** clause may be specified only for a FUNCTION, for which it is mandatory.
- It indicates the return type of the function, and the function body must contain a RETURN value statement.

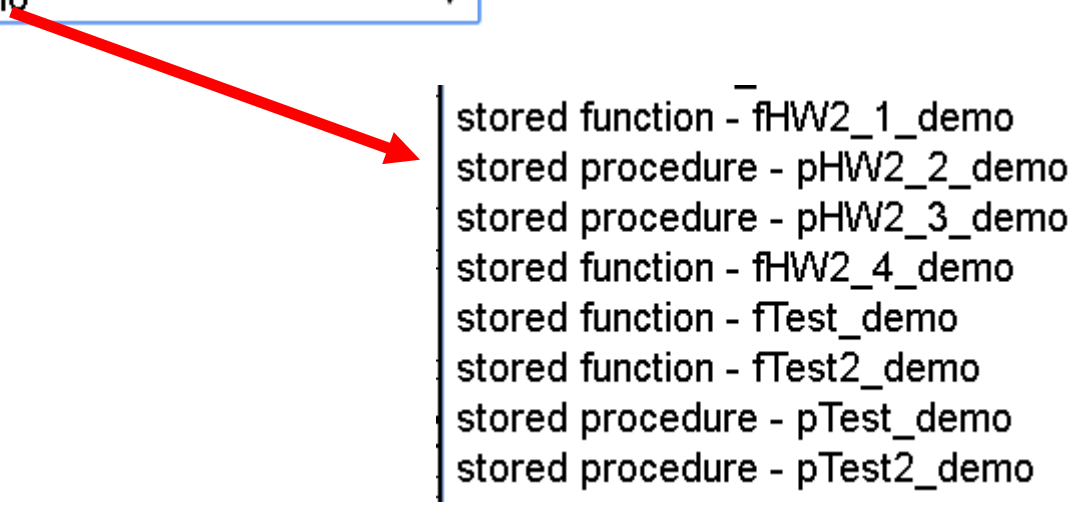
Exercises on the class website



Homework

5. You have to manually create the table/view/procedure on **CPS3740_2017S** database through MySQL command-line before submit below functions.
   - 1. Get permission to execute,drop the selected function. (only for function.)
   - 2. Get permission to execute,drop the selected procedure. (Only for procedure.)
   - 3. Get permission to show view,alter,insert,update,delete,drop the selected table/view. (NOT for procedure.)

   Please select your table name to have grant,drop,update,insert,delete,trigger and alter privileges: table - Test_demo ▼



MySQL Stored Procedure example 1

(1) Create Store Procedure

```
DELIMITER //
CREATE PROCEDURE pTest_xxxx()
BEGIN
  SELECT * FROM dreamhome.Users;
END //
DELIMITER ;      -- change the delimiter back to ;
```

(2) Get permission

Get permission on pTest_xxxx on the class website, opt 6.2

(3) Call/Drop Store Procedure

```
Mysql> CALL pTest_xxxx();
Mysql> drop procedure pTest_xxxx;
```

MySQL Stored Procedure example 2

```
DELIMITER //
CREATE PROCEDURE pTest_xxxx(IN n CHAR(20))
BEGIN
  SELECT * FROM dreamhome.Users WHERE login= n;
END //
DELIMITER ;      -- change the delimiter back to ;

mysql> CALL pTest_xxxx(‘tiger');
mysql> drop procedure pTest_xxxx ;
```

LIKE in Stored Procedure

```
DELIMITER //
CREATE PROCEDURE pTest_xxxx(IN n CHAR(20))
BEGIN
  SELECT * FROM dreamhome.Users
  WHERE login LIKE CONCAT('%',n, '%');
END //
DELIMITER ;

mysql> CALL pTest_xxxx('u');
mysql> drop procedure pTest_xxxx ;
```

Stored Procedure vs Function 1

(1) Create Store Function

```
DELIMITER //
CREATE FUNCTION fTest_xxxx(p int)
  RETURNS INT
BEGIN
  DECLARE total INT DEFAULT 0;
  SELECT COUNT(quantity*price) INTO total
  FROM dreamhome.Product WHERE price > p;
  RETURN total;
END //
DELIMITER ;     -- change the delimiter back to ;
```

(2) Get permission

Get permission on pTest_xxxx on the class website, opt 6.1

(3) Call/Drop Store Function

```
mysql> select fTest_xxxx(2);
mysql> drop function fTest_xxxx;
```

```
+---------------+
| fTest_demo(2) |
+---------------+
|             1 |
+---------------+
```

Stored Procedure vs Function 2

```
DELIMITER //
CREATE PROCEDURE pTest_xxxx(IN p int)
BEGIN
  DECLARE total INT DEFAULT 0;
  SELECT COUNT(id) as ct, sum(quantity*price) as total
  FROM dreamhome.Product WHERE price > p; END //
DELIMITER ;     -- change the delimiter back to ;
```

```
+----+-------+
| ct | total |
+----+-------+
|  1 |     4 |
+----+-------+
```

```
mysql> call pTest_demo(2);
mysql> drop procedure pTest_demo;
```

Stored Procedure vs Function 3

```
DELIMITER //
CREATE PROCEDURE pTest_xxxx (IN p int, OUT t int)
BEGIN
  DECLARE total INT DEFAULT 0;
  SELECT COUNT(quantity*price) into total
  FROM dreamhome.Products WHERE price > p;
  set t = total;
END //
DELIMITER ;      -- change the delimiter back to ;
```

```
mysql> call pTest_xxxx (10,@total);
mysql> select @total;
```

```
+--------+
| @total |
+--------+
|     28 |
+--------+
```

DROP routines before CREATE

```
DELIMITER $$
DROP FUNCTION IF EXISTS fTest_xxxxx ;
CREATE FUNCTION fTest_xxxx(branchno varchar(12))
  RETURNS float
     BEGIN
……..SQL statements…….
    END $$
 DELIMITER ;
```

Exercise – Stored Function

- Write a simple stored function that
  - Takes a gender as an input
  - Find the staff whose gender matches the input
  - Return the results
- Write a simple stored procedure that
  - Takes a gender as an input
  - Find the staff whose gender matches the input
  - Print the results

Conclusion

- Understand SQL variables
- Write and run a simple stored function
- Write and run a simple stored procedure